DCC042 – Robótica Móvel

Roteiro do Laboratório 1 — TurtleBot3

Comandos de velocidade e desvio básico de obstáculos com LiDAR

Este roteiro está dividido em duas partes: (1) preparação do ambiente, a ser feita ANTES da aula de laboratório, e (2) a atividade prática, realizada em sala com o robô físico.

> **Antes de vir para o laboratório**
> Complete integralmente a Parte 1 (Preparação) em seu próprio computador. Alunos que chegarem ao laboratório sem o ambiente testado poderão não conseguir concluir a atividade no tempo de aula.

Parte 1 — Preparação do ambiente (fazer antes da aula)

1.1 Visão geral

Vamos controlar o TurtleBot3 remotamente, a partir do seu notebook, sem precisar instalar o framework ROS2. A comunicação com o robô é feita via **rosbridge**: o robô expõe uma ponte WebSocket, e nosso lado (o notebook) conversa com essa ponte usando a biblioteca Python **roslibpy**. Você não precisa entender ROS2 internamente para esta disciplina, só usar a API Python que vamos fornecer.

1.2 Requisitos

- Python 3.8 ou superior instalado (verifique com *python3 --version* no terminal).
- Editor de código ou Jupyter Notebook (o mesmo ambiente que vocês já usam com o CoppeliaSim serve).
- Notebook conectado à rede Wi-Fi que será disponibilizada no laboratório (informações de rede serão passadas no início da aula).

1.3 Instalação da biblioteca roslibpy

Abra um terminal e execute:

```
pip install roslibpy
```

Se você utiliza ambientes virtuais (recomendado), ative o ambiente do curso antes de instalar:

```
conda activate robotica-movel
pip install roslibpy
```

Verifique a instalação:

```
python3 -c "import roslibpy; print(roslibpy.__version__)"
```

Se aparecer um número de versão sem erros, está tudo certo.

1.4 Teste de conexão (sem o robô)

Como o robô só estará disponível apenas no dia do laboratório, este teste apenas confirma que a biblioteca está funcionando (a conexão real será testada em sala). Execute o script abaixo; é esperado que ele NÃO conecte (pois o robô está desligado/fora da rede) e apresente um erro ou timeout após alguns segundos. Isso é normal e apenas confirma que o roslibpy está instalado corretamente.

```
import roslibpy

# IP de exemplo — o IP real do robô será informado em sala
client = roslibpy.Ros(host='192.168.0.100', port=9090)

try:
    client.run(timeout=5)
    print('Conectado:', client.is_connected)
except Exception as e:
    print('Sem conexão (esperado fora do laboratório):', e)
```

1.5 Leitura recomendada antes da aula

- Revisar o conceito de mensagens Twist (velocidade linear e angular) — usado para comandar o robô.
- Revisar o conceito de LaserScan — formato do dado de um sensor de varredura a laser (lista de distâncias por ângulo).
- Reler a seção de Sensores do material da disciplina (Aula 17).

Checklist antes de sair de casa

[ ] pip install roslibpy executado sem erro
[ ] script de teste da Seção 1.4 executado
[ ] notebook carregado e com bateria

Parte 2 — Atividade prática em laboratório

2.1 Objetivo

Nesta atividade você irá: (1) estabelecer conexão com o robô físico via roslibpy; (2) enviar comandos de velocidade básicos e observar o movimento do robô; (3) implementar um desviador de obstáculos simples usando os dados do LiDAR.

2.2 Informações fornecidas em sala

- IP do robô designado ao seu grupo (cada robô tem um IP fixo — anote o seu).
- Porta de conexão: 9090 (padrão do rosbridge).
- Confirmação de que o robô está ligado e o rosbridge_server está ativo (indicado pelo monitor/professor).

2.3 Etapa 1 — Conectar ao robô

Adapte o IP abaixo para o IP do robô do seu grupo e execute:

```
import roslibpy
import time

ROBOT_IP = '192.168.0.XXX'   # <-- substitua pelo IP do seu robô
PORT = 9090

client = roslibpy.Ros(host=ROBOT_IP, port=PORT)
client.run()

print('Conectado:', client.is_connected)
```

Não conectou?

Confira: (1) está na mesma rede Wi-Fi do laboratório; (2) o IP está correto; (3) o robô foi ligado pelo monitor. Chame o professor/monitor se o problema persistir — não gaste muito tempo tentando sozinho.

2.4 Etapa 2 — Enviar comandos de velocidade

O robô é controlado publicando mensagens do tipo geometry_msgs/Twist no tópico /cmd_vel. Execute o código abaixo em um espaço aberto e observe o comportamento do robô.

```
cmd_vel = roslibpy.Topic(client, '/cmd_vel', 'geometry_msgs/Twist')

def move(v_linear, v_angular, duracao):
    msg = roslibpy.Message({
        'linear': {'x': v_linear, 'y': 0.0, 'z': 0.0},
        'angular': {'x': 0.0, 'y': 0.0, 'z': v_angular}
    })
    cmd_vel.publish(msg)
    time.sleep(duracao)
    parar()

def parar():
    cmd_vel.publish(roslibpy.Message({
        'linear': {'x': 0.0, 'y': 0.0, 'z': 0.0},
        'angular': {'x': 0.0, 'y': 0.0, 'z': 0.0}
    }))
```

```
# Teste 1: andar para frente por 2 segundos
move(v_linear=0.1, v_angular=0.0, duracao=2)

# Teste 2: girar no próprio eixo por 2 segundos
move(v_linear=0.0, v_angular=0.5, duracao=2)
```

Tarefa 2.4.1: modifique os valores de v_linear e v_angular e observe como o robô responde. Registre no relatório: qual foi a velocidade linear máxima em que o robô ainda manteve trajetória estável em linha reta?

2.5 Etapa 3 — Ler dados do LiDAR

O robô publica leituras do sensor LiDAR no tópico /scan, do tipo sensor_msgs/LaserScan. O campo ranges é uma lista de distâncias (em metros), uma para cada ângulo de varredura.

```
scan_data = {}

def callback_scan(msg):
    scan_data['ranges'] = msg['ranges']
    scan_data['angle_min'] = msg['angle_min']
    scan_data['angle_increment'] = msg['angle_increment']

scan = roslibpy.Topic(client, '/scan', 'sensor_msgs/LaserScan')
scan.subscribe(callback_scan)

time.sleep(1)  # aguardar a primeira leitura chegar
print('Número de leituras:', len(scan_data['ranges']))
print('Distância à frente (índice 0):', scan_data['ranges'][0])
```

Tarefa 2.5.1: identifique, no vetor ranges, o índice aproximado correspondente à leitura diretamente à frente do robô (0°) e à leitura diretamente atrás (180°). Dica: use angle_min e angle_increment para converter índice em ângulo.

2.6 Etapa 4 — Desvio básico de obstáculos

Combine as etapas anteriores para implementar um comportamento reativo simples: o robô anda para frente e, ao detectar um obstáculo próximo à sua frente, para e gira até a direção frontal estar livre novamente.

```
import math

DISTANCIA_SEGURANCA = 0.4  # metros
VELOCIDADE_FRENTE = 0.1
VELOCIDADE_GIRO = 0.4

def distancia_frontal():
    ranges = scan_data.get('ranges', [])
    if not ranges:
        return float('inf')
    # janela pequena em torno do índice frontal (ajuste conforme seu robô)
    frente = ranges[0:5] + ranges[-5:]
    validos = [d for d in frente if d > 0.0 and not math.isinf(d)]
    return min(validos) if validos else float('inf')

def desviar_obstaculos(tempo_total=20):
    inicio = time.time()
    while time.time() - inicio < tempo_total:
        d = distancia_frontal()
```

```
        if d < DISTANCIA_SEGURANCA:
            cmd_vel.publish(roslibpy.Message({
                'linear': {'x': 0.0, 'y': 0.0, 'z': 0.0},
                'angular': {'x': 0.0, 'y': 0.0, 'z': VELOCIDADE_GIRO}
            }))
        else:
            cmd_vel.publish(roslibpy.Message({
                'linear': {'x': VELOCIDADE_FRENTE, 'y': 0.0, 'z': 0.0},
                'angular': {'x': 0.0, 'y': 0.0, 'z': 0.0}
            }))
        time.sleep(0.1)
    parar()

desviar_obstaculos()
```

Tarefa 2.6.1: execute o comportamento acima em um espaço com pelo menos um obstáculo. O robô conseguiu desviar de forma consistente? Ajuste DISTANCIA_SEGURANCA e VELOCIDADE_GIRO se necessário e registre os valores finais utilizados.

Tarefa 2.6.2 (opcional, se sobrar tempo): em vez de girar sempre para o mesmo lado, modifique a lógica para comparar as leituras do lado esquerdo e direito do robô e girar para o lado com maior distância livre.

2.7 Encerramento

Ao finalizar os testes, sempre pare o robô e encerre a conexão:

```
parar()
client.terminate()
```

Antes de sair

Devolva o robô à posição/estado indicado pelo monitor e confirme que o comando parar() foi executado com sucesso.

2.8 O que entregar

1. Respostas às Tarefas 2.4.1, 2.5.1 e 2.6.1 (e 2.6.2, se realizada).
2. Trecho de código final utilizado na Etapa 4, com os valores de parâmetros ajustados pelo grupo.
3. Um breve parágrafo (5–8 linhas) descrevendo dificuldades encontradas e como foram resolvidas.